

FITBUDDY AI

Phase 5: Project Development Phase

Prepared from the current public FitBuddy_ai repository and organized according to the Phase 1–6 structure of the provided AI/ML & GenAI project template.

5.1 Development Status

The current public FitBuddy_ai repository contains the core FastAPI/SQLite web application. The implementation visible in routes.py handles registration, workout-plan construction, database persistence, individual user retrieval, feedback-based update records and an administrative user listing.

5.2 Current Source Structure

File	Role in implementation
main.py	FastAPI application entry point and database startup initialization.
routes.py	Core application workflow and HTTP endpoints.
database.py	SQLite schema and connection handling.
config.py	Environment-backed settings including Gemini model names.
index.html	Profile registration form.
dashboard.html	User profile, plan, nutrition tip and feedback interface.
admin.html	Administrative registry of all user records.
fitbuddy.db	SQLite database file present in the repository.

5.3 Implemented Workflow

- 1 The user opens the FitBuddy home page.
- 2 The user enters name, age, gender, weight, height, workout days, objective and experience level.
- 3 POST /generate-plan receives the form data.
- 4 The current route constructs a workout blueprint using the submitted profile values.
- 5 The profile and original plan are inserted into the users table.
- 6 The dashboard displays the stored profile and active plan.
- 7 The user submits feedback using POST /submit-feedback.
- 8 The route retrieves the existing record and stores an adapted plan plus feedback text.
- 9 The updated dashboard displays the revised plan and the original baseline.
- 10 GET /users provides the administrator registry.

5.4 Current Workout Generation Logic

The visible implementation currently builds a text-based workout schedule directly in routes.py. The route includes conditional content for different workout-day counts and stores the resulting schedule as original_plan. This is

functional application logic, but it is not the same as a live Gemini API call.

5.5 Gemini Integration Readiness

config.py defines a required `gemini_api_key` and configurable workout/tip model names loaded from `.env`. This establishes configuration readiness. For the application to become genuinely Gemini-driven, a dedicated service module should call the configured model, pass validated profile information, parse the response and handle API failures. The current public route does not visibly perform that call.

5.6 Feedback Module

The feedback pipeline retrieves the original record, prepends the submitted feedback to the baseline plan, stores the result in `updated_plan`, and stores `feedback_text`. This provides the database structure needed for future intelligent feedback interpretation.

5.7 Future Development Items

- Move workout generation into a dedicated Gemini service module.
- Add a separate nutrition/recovery generation service.
- Use structured JSON output or a validated schema for model responses.
- Add stronger validation for age, height, weight and workout-day ranges.
- Add automated tests for routes and database operations.
- Keep API secrets out of Git history and public files.
- Improve the feedback updater so feedback changes the schedule rather than only being prepended to it.

5.8 Development Deliverable

The development phase deliverable is the runnable FitBuddy web application together with its source files, database schema, HTML templates and configuration. The public repository currently provides the core application needed to demonstrate the registration → plan → feedback → updated plan → admin flow.

Project repository: https://github.com/asvikamoorthy-cyber/FitBuddy_ai

Reference structure: <https://github.com/Ravi-teja-777/AI-ML-and-GEN-AI-Track-Project-Template>

Scope note: This document describes the project state visible in the public repository and identifies proposed/future items separately where applicable.